

Codyyssey_A1-3

배포 URL

<https://codyyssey-a1-3-egmw-k2cxrcoeo-jmkm0520.vercel.app>

GitHub 저장소

https://github.com/jmkm0520-glitch/Codyyssey_A1-3

AI 웹사이트 프로젝트 예상 면접 답변

1. HTML / CSS / JS / api 구조를 왜 이렇게 나눴는가?

프로젝트를 기능별로 관리하기 쉽도록 역할을 나누었습니다.

- **HTML** : 웹페이지의 구조를 만드는 역할입니다.
- **CSS** : 화면의 디자인과 레이아웃을 꾸미는 역할입니다.
- **JavaScript(JS)** : 버튼 클릭, 입력값 처리, AI 호출 등 사용자와의 상호작용을 담당합니다.
- **api/** : AI API를 호출하는 백엔드(Serverless Function)입니다. API 키를 안전하게 보관하고 AI와 통신하는 역할을 합니다.

이렇게 역할을 분리하면 코드가 깔끔해지고, 수정이나 유지보수가 쉬워집니다.

2. AI 기능에서 로딩 / 성공 / 실패 상태를 어떻게 처리했는가?

AI는 응답을 받기까지 시간이 걸리기 때문에 사용자에게 현재 상태를 알려주도록 구현했습니다.

① 로딩

사용자가 "추천받기" 버튼을 누르면

| "AI가 레시피를 생성하고 있습니다..."

라는 메시지를 보여줍니다.

사용자는 프로그램이 정상적으로 동작 중이라는 것을 알 수 있습니다.

② 성공

AI 응답을 정상적으로 받으면

추천 레시피를 화면에 출력합니다.

③ 실패

다음과 같은 경우에는 오류 메시지를 출력했습니다.

- 재료를 입력하지 않은 경우
- API 호출 실패
- 서버 오류 발생

예시

| "재료를 입력해주세요."

또는

| "잠시 후 다시 시도해주세요."

3. Serverless Function에서 입력 검증과 응답 포맷을 어떻게 구성했는가?

먼저 사용자가 재료를 입력했는지 확인했습니다.

빈 값이면 AI를 호출하지 않고 바로 오류 메시지를 반환했습니다.

예시

```
{  
  "error": "재료를 입력해주세요."  
}
```

정상 입력이면 Gemini API를 호출한 뒤

JSON 형태로 결과를 프론트에 전달했습니다.

예시

```
{
  "recipe": "김치볶음밥 ..."
}
```

JSON 형식을 사용한 이유는 프론트엔드에서 데이터를 쉽게 사용할 수 있기 때문입니다.

4. 배포 후 발생한 문제를 어떻게 해결했는가?

문제가 발생하면 다음 순서대로 확인했습니다.

① 브라우저 개발자 도구(Console)

↓

② Network 탭에서 API 요청 확인

↓

③ Vercel Function Logs 확인

↓

④ 코드 수정

↓

⑤ GitHub Push

↓

⑥ Vercel 자동 재배포

↓

⑦ 다시 테스트

실제로 API 오류와 모델 오류를 이런 순서로 해결했습니다.

5. HTML / CSS / JavaScript의 역할 차이를 설명하시오.

이번 프로젝트를 예로 들면

HTML

재료 입력창

추천 버튼

결과 영역

등 화면의 구조를 만들었습니다.

CSS

색상

간격

버튼 디자인

모바일 반응형

등 화면을 꾸몄습니다.

JavaScript

재료 입력값을 읽고

버튼 클릭 이벤트를 처리하며

fetch()를 통해 AI를 호출하고

응답을 화면에 출력했습니다.

6. fetch 요청이 서버리스 함수로 전달되고 응답이 돌아오는 흐름

동작 순서는 다음과 같습니다.

사용자

↓

재료 입력

↓

추천 버튼 클릭

↓

JavaScript에서

```
fetch("/api/recommend")
```

실행

↓

Serverless Function 실행

↓

Gemini API 호출

↓

Gemini 응답 수신

↓

JSON 반환

↓

JavaScript가 결과를 받아 화면에 출력

즉,

프론트엔드와 AI 사이를 Serverless Function이 연결해주는 구조입니다.

7. 환경 변수를 사용하는 이유

API Key를 코드에 직접 작성하면

GitHub에 올라갈 수 있고

누군가 사용할 위험이 있습니다.

그래서

Vercel의 Environment Variables에 저장했습니다.

장점

- API Key 보호
- 키 변경이 쉬움
- GitHub에 노출되지 않음

보안을 위해 반드시 필요한 방법입니다.

8. AI 기능을 넣은 이유와 프롬프트를 어떻게 구성했는가?

서비스의 목적은

사용자가 냉장고에 있는 재료만으로 만들 수 있는 음식을 추천받는 것입니다.

그래서 AI가 가장 적합하다고 생각했습니다.

프롬프트에는

- 사용자가 입력한 재료
- 만들 수 있는 음식 추천
- 조리 방법
- 필요한 추가 재료

등을 포함하도록 작성했습니다.

AI가 단순히 음식 이름만 알려주는 것이 아니라

실제로 따라 만들 수 있는 레시피를 제공하도록 구성했습니다.

9. 응답이 느리다면 어떤 개선 방법을 사용할 것인가?

응답 속도가 느리다면

다음 방법을 고려할 수 있습니다.

- 더 빠른 AI 모델 사용
- 프롬프트를 짧게 작성
- 결과 길이 제한
- 동일한 요청은 캐시 사용
- 로딩 애니메이션 제공
- 요청 횟수 제한

이렇게 하면 속도와 비용을 함께 줄일 수 있습니다.

10. AI 기능을 2개로 늘린다면?

현재는

```
/api/recommend
```

하나만 있습니다.

AI 기능이 늘어나면

예를 들어

```
/api/recommend  
/api/nutrition  
/api/ingredient
```

처럼 기능별 API를 분리하겠습니다.

프론트에서도

각 버튼이

각각 다른 API를 호출하도록 구성하면 유지보수가 쉬워집니다.

11. API Key가 유출되었다면?

가장 먼저

Google AI Studio에서

API Key를 삭제(폐기)합니다.

그 후

새 API Key를 발급받아

Vercel Environment Variables를 수정합니다.

GitHub 커밋에도 노출되었다면

커밋 이력을 정리하거나

새 Key로 교체합니다.

앞으로는

절대 코드나 GitHub에 API Key를 작성하지 않습니다.

12. 프론트 프레임워크(React 등)를 사용할 수 있다면?

현재는

HTML

CSS

JavaScript만 사용했습니다.

React를 사용하면

장점

- 컴포넌트 재사용
- 상태 관리가 편리
- 화면 업데이트가 쉬움
- 프로젝트가 커져도 관리하기 좋음

단점

- 초기 학습이 필요
- 프로젝트 구조가 복잡해짐
- 빌드 과정이 추가됨

변경해야 하는 부분은

HTML과 JavaScript를 React 컴포넌트로 변경하고,

페이지 이동은 React Router,

상태 관리는 React의 state를 사용하는 방식으로 바꿉니다.

Serverless Function(API)는 그대로 사용할 수 있기 때문에 백엔드는 크게 변경하지 않아도 됩니다.